

Exercise 1: Implementing the Singleton Pattern

Logger.java

```
class Logger
{
    private static Logger instance;
    private Logger() {
        System.out.println("Logger Created");
    }

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String msg) {
        System.out.println("LOG: " + msg);
    }
}
```

SingletonTest.java

```
public class SingletonTest
{
    public static void main(String[] args)
    {
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();
        logger1.log("Application Started");
        if (logger1 == logger2) {
            System.out.println("Only One Instance Exists");
        }
    }
}
```

Output

```
Logger Created
LOG: Application Started
Only One Instance Exists
```

Exercise 2: Implementing the Factory Method Pattern

Document.java

```
interface Document {  
    void open();  
}
```

WordDocument.java

```
class WordDocument implements Document {  
    public void open() {  
        System.out.println("Word Document Opened");  
    }  
}
```

PdfDocument.java

```
class PdfDocument implements Document {  
    public void open() {  
        System.out.println("PDF Document Opened");  
    }  
}
```

ExcelDocument.java

```
class ExcelDocument implements Document {  
    public void open() {  
        System.out.println("Excel Document Opened");  
    }  
}
```

DocumentFactory.java

```
abstract class DocumentFactory {  
    abstract Document createDocument();  
}
```

Factories

```
class WordFactory extends DocumentFactory {  
    Document createDocument() {  
        return new WordDocument();  
    }  
}
```

```

    }
}
class PdfFactory extends DocumentFactory {
    Document createDocument() {
        return new PdfDocument();
    }
}
class ExcelFactory extends DocumentFactory {
    Document createDocument() {
        return new ExcelDocument();
    }
}
}

```

FactoryTest.java

```

public class FactoryTest {
    public static void main(String[] args) {
        DocumentFactory factory =
            new PdfFactory()
        Document doc =
            factory.createDocument();
        doc.open();
    }
}

```

Output

PDF Document Opened

Exercise 3: Implementing the Builder Pattern

Computer.java

```

class Computer
    private String cpu;
    private int ram;
    private int storage;
    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
    }
    public void display() {
        System.out.println( cpu + " " + ram + "GB " + storage + "GB");
    }
}

```

```

    }
    static class Builder {
        private String cpu;
        private int ram;
        private int storage;
        public Builder setCPU(String cpu) {
            this.cpu = cpu;
            return this;
        }
        public Builder setRAM(int ram) {
            this.ram = ram;
            return this;
        }
        public Builder setStorage(int storage) {
            this.storage = storage;
            return this;
        }
        public Computer build() {
            return new Computer(this);
        }
    }
}

```

BuilderTest.java

```

public class BuilderTest {
    public static void main(String[] args) {
        Computer pc =
            new Computer.Builder()
                .setCPU("i7")
                .setRAM(16)
                .setStorage(512)
                .build();
        pc.display();
    }
}

```

Output

i7 16GB 512GB

Exercise 4: Implementing the Adapter Pattern

PaymentProcessor.java

```
interface PaymentProcessor {  
    void processPayment(double amount);  
}
```

Payment Gateways

```
class PayPalGateway {  
    public void sendPayment(double amount) {  
        System.out.println("Paid ₹" + amount + " using PayPal");  
    }  
}  
  
class StripeGateway {  
    public void makePayment(double amount) {  
        System.out.println("Paid ₹" + amount + " using Stripe");  
    }  
}
```

Adapters

```
class PayPalAdapter implements PaymentProcessor {  
    private PayPalGateway gateway;  
    public PayPalAdapter(PayPalGateway gateway) {  
        this.gateway = gateway;  
    }  
    public void processPayment(double amount) {  
        gateway.sendPayment(amount);  
    }  
}  
  
class StripeAdapter implements PaymentProcessor {  
    private StripeGateway gateway;  
    public StripeAdapter(StripeGateway gateway) {  
        this.gateway = gateway;  
    }  
    public void processPayment(double amount) {  
        gateway.makePayment(amount);  
    }  
}
```

AdapterTest.java

```
public class AdapterTest {
    public static void main(String[] args) {
        PaymentProcessor p1 =
            new PayPalAdapter(new PayPalGateway());
        PaymentProcessor p2 =
            new StripeAdapter(new StripeGateway());
        p1.processPayment(500);
        p2.processPayment(1000);
    }
}
```

Output

Paid ₹500.0 using PayPal
Paid ₹1000.0 using Stripe

Exercise 5: Implementing the Decorator Pattern

Notifier.java

```
interface Notifier {
    void send(String message);
}
```

EmailNotifier.java

```
class EmailNotifier implements Notifier {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}
```

NotifierDecorator.java

```
abstract class NotifierDecorator
    implements Notifier {
    protected Notifier notifier;
    public NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }
    public void send(String message) {
        notifier.send(message);
    }
}
```

```
}  
}
```

SMS Decorator

```
class SMSNotifierDecorator  
    extends NotifierDecorator  
    public SMSNotifierDecorator(  
        Notifier notifier) {  
        super(notifier);  
    }  
    public void send(String message) {  
        super.send(message);  
        System.out.println("SMS: " + message);  
    }  
}
```

Slack Decorator

```
class SlackNotifierDecorator  
    extends NotifierDecorator  
    public SlackNotifierDecorator(  
        Notifier notifier) {  
        super(notifier);  
    }  
    public void send(String message) {  
        super.send(message);  
        System.out.println("Slack: " + message);  
    }  
}
```

DecoratorTest.java

```
public class DecoratorTest {  
    public static void main(String[] args) {  
        Notifier notifier =  
            new SlackNotifierDecorator(  
                new SMSNotifierDecorator(  
                    new EmailNotifier()  
                )  
            );  
        notifier.send("Meeting at 10 AM");  
    }  
}
```

Output

Email: Meeting at 10 AM

SMS: Meeting at 10 AM

Slack: Meeting at 10 AM

Exercise 6: Implementing the Proxy Pattern

Image.java

```
interface Image {  
    void display();  
}
```

RealImage.java

```
class RealImage implements Image {  
    private String fileName;  
    public RealImage(String fileName) {  
        this.fileName = fileName;  
        loadFromServer();  
    }  
    private void loadFromServer() {  
        System.out.println(  
            "Loading " + fileName + " from server..."  
        );  
    }  
    public void display() {  
        System.out.println(  
            "Displaying " + fileName  
        );  
    }  
}
```

ProxyImage.java

```
class ProxyImage implements Image {  
    private RealImage realImage;  
    private String fileName;  
    public ProxyImage(String fileName) {  
        this.fileName = fileName;  
    }  
    public void display() {
```



```
        if(realImage == null) {
            realImage =
                new RealImage(fileName);
        }
        realImage.display();
    }
}
```

ProxyTest.java

```
public class ProxyTest {
    public static void main(String[] args) {
        Image image =
            new ProxyImage("nature.jpg");
        System.out.println("Image Created");
        image.display();
        image.display();
    }
}
```

Output

Image Created
Loading nature.jpg from server...
Displaying nature.jpg
Displaying nature.jpg

Exercise 7: Implementing the Observer Pattern

Observer.java

```
interface Observer {
    void update(double price);
}
```

Stock.java

```
interface Stock {
    void registerObserver(Observer o);
    void removeObserver(Observer o);
    void notifyObservers();
}
```

StockMarket.java

```
import java.util.*;
class StockMarket implements Stock {
    private List<Observer> observers =
        new ArrayList<>();
    private double stockPrice;
    public void registerObserver(Observer o) {
        observers.add(o);
    }
    public void removeObserver(Observer o) {
        observers.remove(o);
    }
    public void notifyObservers() {
        for(Observer o : observers) {
            o.update(stockPrice);
        }
    }
    public void setStockPrice(double price) {
        this.stockPrice = price;
        notifyObservers();
    }
}
```

MobileApp.java

```
class MobileApp implements Observer {
    public void update(double price) {
        System.out.println(
            "Mobile App: Stock Price = " + price
        );
    }
}
```

WebApp.java

```
class WebApp implements Observer {
    public void update(double price) {
        System.out.println(
            "Web App: Stock Price = " + price
        );
    }
}
```

ObserverTest.java

```
public class ObserverTest {  
    public static void main(String[] args) {  
        StockMarket market =  
            new StockMarket()  
        Observer mobile =  
            new MobileApp();  
        Observer web =  
            new WebApp();  
        market.registerObserver(mobile);  
        market.registerObserver(web);  
        market.setStockPrice(1500);  
    }  
}
```

Output

Mobile App: Stock Price = 1500.0
Web App: Stock Price = 1500.0

Exercise 8: Implementing the Strategy Pattern

PaymentStrategy.java

```
interface PaymentStrategy {  
    void pay(int amount);  
}
```

CreditCardPayment.java

```
class CreditCardPayment  
    implements PaymentStrategy {  
    public void pay(int amount) {  
        System.out.println(  
            "Paid " + amount +  
            " using Credit Card"  
        );  
    }  
}
```

PayPalPayment.java

```
class PayPalPayment
    implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println(
            "Paid " + amount +
            " using PayPal"
        );
    }
}
```

PaymentContext.java

```
class PaymentContext {
    private PaymentStrategy strategy;
    public PaymentContext(
        PaymentStrategy strategy) {
        this.strategy = strategy;
    }
    public void executePayment(
        int amount) {
        strategy.pay(amount);
    }
}
```

StrategyTest.java

```
public class StrategyTest {
    public static void main(String[] args) {
        PaymentContext payment;
        payment =
            new PaymentContext(
                new CreditCardPayment());
        payment.executePayment(1000);
        payment =
            new PaymentContext(
                new PayPalPayment());
        payment.executePayment(2000);
    }
}
```

Output

Paid 1000 using Credit Card

Paid 2000 using PayPal

Exercise 9: Implementing the Command Pattern

Command.java

```
interface Command {  
    void execute();  
}
```

Light.java (Receiver)

```
class Light {  
    public void on() {  
        System.out.println("Light ON");  
    }  
    public void off() {  
        System.out.println("Light OFF");  
    }  
}
```

LightOnCommand.java

```
class LightOnCommand  
    implements Command {  
    private Light light;  
    public LightOnCommand(  
        Light light) {  
        this.light = light;  
    }  
    public void execute() {  
        light.on();  
    }  
}
```

LightOffCommand.java

```
class LightOffCommand  
    implements Command {  
    private Light light;
```

```

    public LightOffCommand(
        Light light) {
        this.light = light;
    }
    public void execute() {
        light.off();
    }
}

```

RemoteControl.java (Invoker)

```

class RemoteControl {
    private Command command;
    public void setCommand(
        Command command) {
        this.command = command;
    }
    public void pressButton() {
        command.execute();
    }
}

```

CommandTest.java

```

public class CommandTest {
    public static void main(String[] args) {
        Light light = new Light();
        Command on =
            new LightOnCommand(light);
        Command off =
            new LightOffCommand(light);
        RemoteControl remote =
            new RemoteControl();
        remote.setCommand(on);
        remote.pressButton();
        remote.setCommand(off);
        remote.pressButton();
    }
}

```

Exercise 10: Implementing the MVC Pattern

Student.java (Model)

```
class Student {
    private String name;
    private int id;
    private String grade;
    public Student(String name, int id, String grade) {
        this.name = name;
        this.id = id;
        this.grade = grade;
    }
    public String getName() {
        return name;
    }
    public int getId() {
        return id;
    }
    public String getGrade() {
        return grade;
    }
    public void setName(String name) {
        this.name = name;
    }
    public void setGrade(String grade) {
        this.grade = grade;
    }
}
```

StudentView.java (View)

```
class StudentView {
    public void displayStudentDetails(
        String name,
        int id,
        String grade) {
        System.out.println("Student Details");
        System.out.println("Name : " + name);
        System.out.println("ID   : " + id);
        System.out.println("Grade: " + grade);
    }
}
```

StudentController.java (Controller)

```
class StudentController {
    private Student model;
    private StudentView view;
    public StudentController(
        Student model,
        StudentView view) {
        this.model = model;
        this.view = view;
    }
    public void setStudentName(String name) {
        model.setName(name);
    }
    public void setStudentGrade(String grade) {
        model.setGrade(grade);
    }
    public void updateView() {
        view.displayStudentDetails(
            model.getName(),
            model.getId(),
            model.getGrade()
        );
    }
}
```

MVCTest.java

```
public class MVCTest {
    public static void main(String[] args) {
        Student model =
            new Student(
                "Keerthan",
                101,
                "A"
            );
        StudentView view =
            new StudentView();
        StudentController controller =
            new StudentController(
                model,
                view
            );
        controller.updateView();
        controller.setStudentGrade("A+");
        System.out.println("\nUpdated Details");
    }
}
```



```
        controller.updateView();
    }
}
```

Output

Student Details
Name : Keerthan
ID : 101
Grade: A

Updated Details
Student Details
Name : Keerthan
ID : 101
Grade: A+

Exercise 11: Implementing Dependency Injection

CustomerRepository.java

```
interface CustomerRepository {
    String findCustomerById(int id);
}
```

CustomerRepositoryImpl.java

```
class CustomerRepositoryImpl
    implements CustomerRepository {
    public String findCustomerById(int id) {
        return "Customer ID : " + id +
            " Name : John";
    }
}
```

CustomerService.java

```
class CustomerService {
    private CustomerRepository repository;
    public CustomerService(
        CustomerRepository repository) {
        this.repository = repository;
    }
}
```

```
}  
public void getCustomer(int id) {  
    System.out.println(  
        repository.findCustomerById(id)  
    );  
}  
}
```

DependencyInjectionTest.java

```
public class DependencyInjectionTest {  
    public static void main(String[] args) {  
        CustomerRepository repository =  
            new CustomerRepositoryImpl();  
        CustomerService service =  
            new CustomerService(repository);  
        service.getCustomer(101);  
    }  
}
```

Output

Customer ID : 101 Name : John